

Commenced Publication in 1973

Founding and Former Series Editors:

Gerhard Goos, Juris Hartmanis, and Jan van Leeuwen

Editorial Board

David Hutchison, UK

Josef Kittler, UK

Alfred Kobsa, USA

John C. Mitchell, USA

Oscar Nierstrasz, Switzerland

Bernhard Steffen, Germany

Demetri Terzopoulos, USA

Gerhard Weikum, Germany

Takeo Kanade, USA

Jon M. Kleinberg, USA

Friedemann Mattern, Switzerland

Moni Naor, Israel

C. Pandu Rangan, India

Madhu Sudan, USA

Doug Tygar, USA

Advanced Research in Computing and Software Science

Subline of Lectures Notes in Computer Science

Subline Series Editors

Giorgio Ausiello, *University of Rome 'La Sapienza', Italy*

Vladimiro Sassone, *University of Southampton, UK*

Subline Advisory Board

Susanne Albers, *University of Freiburg, Germany*

Benjamin C. Pierce, *University of Pennsylvania, USA*

Bernhard Steffen, *University of Dortmund, Germany*

Madhu Sudan, *Microsoft Research, Cambridge, MA, USA*

Deng Xiaotie, *City University of Hong Kong*

Jeannette M. Wing, *Microsoft Research, Redmond, WA, USA*

Kenneth L. McMillan Xavier Rival (Eds.)

Verification, Model Checking, and Abstract Interpretation

15th International Conference, VMCAI 2014
San Diego, CA, USA, January 19-21, 2014
Proceedings

Volume Editors

Kenneth L. McMillan
Microsoft Research
One Microsoft Way
Redmond, WA 98052, USA
E-mail: kenmcmil@microsoft.com

Xavier Rival
DI - Ecole Normale Supérieure
45, rue d'Ulm
75230 Paris Cedex 05, France
E-mail: rival@di.ens.fr

ISSN 0302-9743 e-ISSN 1611-3349
ISBN 978-3-642-54012-7 e-ISBN 978-3-642-54013-4
DOI 10.1007/978-3-642-54013-4
Springer Heidelberg New York Dordrecht London

Library of Congress Control Number: 2013957367

CR Subject Classification (1998): F.3.1-2, D.2.4, C.2.4, F.4.1, F.1.1, I.2.2

LNCS Sublibrary: SL 1 – Theoretical Computer Science and General Issues

© Springer-Verlag Berlin Heidelberg 2014

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed. Exempted from this legal reservation are brief excerpts in connection with reviews or scholarly analysis or material supplied specifically for the purpose of being entered and executed on a computer system, for exclusive use by the purchaser of the work. Duplication of this publication or parts thereof is permitted only under the provisions of the Copyright Law of the Publisher's location, in its current version, and permission for use must always be obtained from Springer. Permissions for use may be obtained through RightsLink at the Copyright Clearance Center. Violations are liable to prosecution under the respective Copyright Law.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

While the advice and information in this book are believed to be true and accurate at the date of publication, neither the authors nor the editors nor the publisher can accept any legal responsibility for any errors or omissions that may be made. The publisher makes no warranty, express or implied, with respect to the material contained herein.

Typesetting: Camera-ready by author, data conversion by Scientific Publishing Services, Chennai, India

Printed on acid-free paper

Springer is part of Springer Science+Business Media (www.springer.com)

Preface

This volume contains the papers presented at VMCAI 2014, the 15th International Conference on Verification, Model Checking, and Abstract Interpretation, held during January 19–21, 2013, in San Diego, co-located with POPL 2014 (the 41st ACM SIGPLAN/SIGACT Symposium on Principles of Programming languages). Previous meetings were held in Port Jefferson (1997), Pisa (1998), Venice (2002), New York (2003), Venice (2004), Paris (2005), Charleston (2006), Nice (2007), San Francisco (2008), Savannah (2009), Madrid (2010), Austin (2011), Philadelphia (2012), and Rome (2013).

VMCAI is a major conference dealing with state-of-the-art research in analysis and verification of programs and systems, with particular emphasis on the cross-fertilization among communities that have developed different methods and models for code and system verification and analysis. VMCAI topics include: program verification, model checking, abstract interpretation and abstract domains, program synthesis, static analysis, type systems, deductive methods, program certification, debugging techniques, program transformation, optimization, hybrid and cyber-physical systems.

This year, we received 64 submissions. Each submission was assessed by at least three Program Committee members. The committee decided to accept 25 papers. We are glad to include in the proceedings the contributions of four invited keynote speakers: Bor-Yuh Evan Chang (University of Colorado, Boulder, USA), Cynthia Dwork (Microsoft Research, USA), Prakash Panangaden (McGill University, Canada) and Thomas Wies (New York University, USA).

We would like to thank all the members of the Program Committee and all the external reviewers for their dedicated effort in evaluating and selecting the papers to be featured in these proceedings. We are also grateful to the Steering Committee for their helpful advice and support. A special thanks goes to Dave Schmidt, who managed the budget of the conference. We would also like to thank Ruzica Piskac who acted as a publicity chair. We would like to thank Suresh Jagannathan, who managed the organization of POPL 2014 as well as David Van Horn, who coordinated the co-located events of POPL.

Finally, we are also grateful to Andrei Voronkov for having set up the Easy-Chair system that was used to handle the submissions, the reviews, the PC discussions and the production workflow.

November 2013

Xavier Rival
Ken McMillan

Organization

Program Committee

Christel Baier	Technische Universität Dresden, Germany
Agostino Cortesi	Università Ca Foscari of Venezia, Italy
Jerome Feret	CNRS and ENS and Inria, France
Alexey Gotsman	IMDEA Software Institute, Spain
Aarti Gupta	NEC Labs, USA
Alan J. Hu	University of British Columbia, Canada
Laura Kovacs	Chalmers University of Technology, Sweden
Daniel Kroening	University of Oxford, UK
Mark Marron	Microsoft Research, USA
Isabella Mastroeni	Università di Verona, Italy
Kenneth L. McMillan	Microsoft Research, USA
Matthew Might	University of Utah, USA
David Monniaux	CNRS and University of Grenoble, France
Kedar Namjoshi	Bell Labs, USA
Peter O'Hearn	Facebook, UK
Ruzica Piskac	Yale University, USA
Sylvie Putot	CEA-LIST, France
Xavier Rival	CNRS and ENS and Inria, France
Philipp Rümmer	Uppsala University, Sweden
Sriram Sankaranarayanan	University of Colorado at Boulder, USA
Tachio Terauchi	Nagoya University, Japan
Tayssir Touili	CNRS and University of Paris Diderot-Paris7, France
Eran Yahav	Technion, Israel

Additional Reviewers

Ahrendt, Wolfgang	Dang, Thao
Blackshear, Sam	Delzanno, Giorgio
Blieberger, Johann	Dimitrov, Dimitar
Bouissou, Olivier	Dimitrova, Rayna
Bucheli, Samuel	Doko, Marko
Bundala, Daniel	Dragan, Ioan
Cachera, David	Fahndrich, Manuel
Cerone, Andrea	Ferns, Norman
D'Oswaldo, Emanuele	Garoché, Pierre-Loïc
Dan, Andrei	Gerke, Michael

Gilray, Thomas
Goubault, Eric
Graf, Susanne
Guan, Nan
Gupta, Ashutosh
Horn, Alexander
Katz, Omer
Khyzha, Artem
Klebanov, Vladimir
Kloos, Johannes
Lammich, Peter
Lampka, Kai
Lauter, Christoph
Lewis, Matt
Liang, Lihao
Liang, Shuying
Lozes, Etienne
Macedo, Hugo
Midtgaard, Jan
Navarro Perez, Juan Antonio
Navas, Jorge A
Paganelli, Gabriele

Peleg, Hila
Poetzl, Daniel
Potet, Marie-Laure
Rinetzky, Noam
Schmitz, Sylvain
Schrammel, Peter
Seghir, Mohamed Nassim
Sharma, Subodh
Song, Fu
Sousa, Marcelo
Sproston, Jeremy
Subotic, Pavle
Suenaga, Kohei
Sutre, Grégoire
Tate, Ross
Tautschnig, Michael
Van Horn, David
Vedrine, Franck
Weissenbacher, Georg
Wies, Thomas
Zwirchmayr, Jakob

Keynote Talks

Minimization of Automata by Duality

Prakash Panangaden

School of Computer Science
McGill University
Montréal, Québec
Canada
`prakash@cs.mcgill.ca`

A remarkable algorithm — discovered by Jan Brzozowski in the early 1960s — is based on the notion of duality. In recent work by Bonchi et al. [1, 2], Bezhanishvili et al. [3] and Dinculescu et al. [4], we have shown how to understand this algorithm as a manifestation of duality. Duality is, strictly speaking, a categorical concept but is widely used and understood much more broadly. In this talk I will explain how duality can be used for the minimization of automata. It turns out to be not just applicable to ordinary automata but also to variations like weighted automata and probabilistic automata of various kinds.

References

1. Bonchi, F., Bonsangue, M.M., Rutten, J.J.M.M., Silva, A.: Brzozowski’s algorithm (Co)Algebraically. In: Constable, R.L., Silva, A. (eds.) *Kozen Festschrift*. LNCS, vol. 7230, pp. 12–23. Springer, Heidelberg (2012)
2. Bonchi, F., Bonsangue, M.M., Hansen, H.H., Panangaden, P., Rutten, J., Silva, A.: Algebra-coalgebra duality in Brzozowski’s minimization algorithm. *ACM Transactions on Computational Logic* (2013)
3. Bezhanishvili, N., Kupke, C., Panangaden, P.: Minimization via duality. In: Ong, L., de Queiroz, R. (eds.) *WoLLIC 2012*. LNCS, vol. 7456, pp. 191–205. Springer, Heidelberg (2012)
4. Dinculescu, M., Hundt, C., Panangaden, P., Pineau, J., Precup, D.: The duality of state and observation in probabilistic transition systems. In: Bezhanishvili, G., Löbner, S., Marra, V., Richter, F. (eds.) *TbiLLC*. LNCS, vol. 7758, pp. 206–230. Springer, Heidelberg (2013)

From Separation Logic to First-Order Logic The Complete Journey^{*}

Thomas Wies

New York University

Abstract. Separation logic (SL) has gained widespread popularity as a formal foundation of tools that analyze and verify heap-manipulating programs. Its great asset lies in its assertion language, which can succinctly express how data structures are laid out in memory, and its discipline of local reasoning, which mimics human intuition about how to prove heap programs correct.

While the succinctness of separation logic makes it attractive for developers of program analysis tools, it also poses a challenge to automation: separation logic is a nonclassical logic that requires specialized theorem provers for discharging the generated proof obligations. SL-based tools therefore implement their own tailor-made theorem provers for this task. However, this brings its own challenges.

The analysis of real-world programs involves more than just reasoning about heap structures. The combination of linked data structures and pointer arithmetic, in particular, is pervasive in low-level system code. Other examples include the dynamic reinterpretation of memory (e.g., treating a memory region both as a linked structure and as an array) and dependencies on data stored in linked structures (e.g., sortedness constraints). To deal with such programs, existing SL tools make simplifying and (deliberately) unsound assumptions about the underlying memory model, rely on interactive help from the user, or implement incomplete extensions to allow some limited support for reasoning about other theories.

The integration of a separation logic prover into an SMT solver can address these challenges. Modern SMT solvers already implement decision procedures for many theories that are relevant in program verification, e.g., linear arithmetic, arrays, and bit-vectors. They also implement generic mechanisms for combining these theories, treating the theory solvers as independent components. These mechanisms provide guarantees about completeness and decidability.

I will present an approach that enables complete combinations of decidable separation logic fragments with other theories in an elegant way. The approach works by reducing SL assertions to first-order logic. The target of this reduction is a decidable fragment of first-order logic that fits well into the SMT framework. That is, reasoning in separation

^{*} This work was supported in part by NSF grant CCS-1320583.

logic is handled entirely by an SMT solver. The approach enables local reasoning about heap programs via a logical encoding of the frame rule and supports complex linked data structures with list and tree-like backbones. The reduction also opens up new possibilities for invariant generation in SL-based static analysis tools (e.g., via the integration of interpolation procedures for first-order logic).

This talk is based on joint work with Ruzica Piskac, Damien Zufferey, and Nishant Totla.

Refuting Heap Reachability

Bor-Yuh Evan Chang

University of Colorado Boulder

`bec@cs.colorado.edu`

Abstract. Precise heap reachability information is a prerequisite for many static verification clients. However, the typical scenario is that the available heap information, computed by say an up-front points-to analysis, is not precise enough for the client of interest. This imprecise heap information in turn leads to a deluge of false alarms for the tool user to triage. Our position is to approach the false alarm problem not just by improving the up-front analysis but by also employing after-the-fact, goal-directed *refutation analyses* that yield targeted precision improvements. We have investigated refutation analysis in the context of detecting statically a class of Android memory leaks. For this client, we have found the necessity for an overall analysis capable of path-sensitive reasoning interprocedurally and with strong updates—a level of precision difficult to achieve globally in an up-front manner. Instead, our approach uses a refutation analysis that mixes highly precise, goal-directed reasoning with facts derived from the up-front analysis to prove alarms false and thus enabling effective and sound filtering of the overall list of alarms.

Table of Contents

SAT-Based Synthesis Methods for Safety Specs	1
<i>Roderick Bloem, Robert Könighofer, and Martina Seidl</i>	
Precise Analysis of Value-Dependent Synchronization in Priority Scheduled Programs	21
<i>Martin D. Schwarz, Helmut Seidl, Vesal Vojdani, and Kalmer Apinis</i>	
Relational Thread-Modular Static Value Analysis by Abstract Interpretation	39
<i>Antoine Miné</i>	
Timing Analysis of Parallel Software Using Abstract Execution	59
<i>Andreas Gustavsson, Jan Gustafsson, and Björn Lisper</i>	
Doomsday Equilibria for Omega-Regular Games	78
<i>Krishnendu Chatterjee, Laurent Doyen, Emmanuel Filiot, and Jean-François Raskin</i>	
Bisimulations and Logical Characterizations on Continuous-Time Markov Decision Processes	98
<i>Lei Song, Lijun Zhang, and Jens Chr. Godskesen</i>	
Probabilistic Automata for Safety LTL Specifications	118
<i>Dileep Kini and Mahesh Viswanathan</i>	
Refuting Heap Reachability	137
<i>Bor-Yuh Evan Chang</i>	
Cascade 2.0	142
<i>Wei Wang, Clark Barrett, and Thomas Wies</i>	
A Logic-Based Framework for Verifying Consensus Algorithms	161
<i>Cezara Drăgoi, Thomas A. Henzinger, Helmut Veith, Josef Widder, and Damien Zufferey</i>	
Verifying Array Programs by Transforming Verification Conditions	182
<i>Emanuele De Angelis, Fabio Fioravanti, Alberto Pettorossi, and Maurizio Proietti</i>	
Weakest Precondition Synthesis for Compiler Optimizations	203
<i>Nuno P. Lopes and José Monteiro</i>	

Message-Passing Algorithms for the Verification of Distributed Protocols	222
<i>Loïc Jezequel and Javier Esparza</i>	
Safety Problems Are NP-complete for Flat Integer Programs with Octagonal Loops	242
<i>Marius Bozga, Radu Iosif, and Filip Konečný</i>	
Parameterized Model Checking of Token-Passing Systems	262
<i>Benjamin Aminof, Swen Jacobs, Ayrat Khalimov, and Sasha Rubin</i>	
Modularly Combining Numeric Abstract Domains with Points-to Analysis, and a Scalable Static Numeric Analyzer for Java	282
<i>Zhoulai Fu</i>	
Generic Combination of Heap and Value Analyses in Abstract Interpretation	302
<i>Pietro Ferrara</i>	
Modeling Parsimonious Putative Regulatory Networks: Complexity and Heuristic Approach	322
<i>Vicente Acuña, Andrés Aravena, Alejandro Maass, and Anne Siegel</i>	
Practical Floating-Point Tests with Integer Code	337
<i>Anthony Romano</i>	
Monitoring Parametric Temporal Logic	357
<i>Peter Faymonville, Bernd Finkbeiner, and Doron Peled</i>	
Precisely Deciding Control State Reachability in Concurrent Traces with Limited Observability	376
<i>Chao Wang and Kevin Hoang</i>	
Modular Synthesis of Sketches Using Models	395
<i>Rohit Singh, Rishabh Singh, Zhilei Xu, Rebecca Krosnick, and Armando Solar-Lezama</i>	
Synthesis with Identifiers	415
<i>Rüdiger Ehlers, Sanjit A. Seshia, and Hadas Kress-Gazit</i>	
Synthesis for Polynomial Lasso Programs	434
<i>Jan Leike and Ashish Tiwari</i>	

Policy Iteration-Based Conditional Termination and Ranking Functions	453
<i>Damien Massé</i>	
Widening for Control-Flow	472
<i>Ben Hardekopf, Ben Wiedermann, Berkeley Churchill, and Vineeth Kashyap</i>	
Author Index	493